



SHAHEED ZULFIKAR ALI BHUTTO
INSTITUTE OF SCIENCE AND TECHNOLOGY

OS LAB PROJECT REPORT

**“MEMORY ALLOCATION SIUMULATOR
(BEST FIT, FIRST FIT, WORST FIT)”**

Prepared By:

Areesha Naeem (2480105)

Submitted To:

Ma'am Bushra

Date:

22-April-2026

1. Introduction:

Memory management is an important concept in an operating system. It is used to allocate memory to the processor efficiently. This project demonstrates 3 memory allocation techniques: First fit, Best fit and Worst fit. These algorithms decide that how memory block is assigned to the processor.

2. Programming Language Used:

This project is developed using the python programming language. For graphic interface Tkinter library is used, in which we create buttons, tables and user interactions.

3. Project Structure

The project is divided into two parts

- Algorithms Logic (First Fit, Best Fit, and Worst Fit)
- Graphical User Interface

The GUI allows users to:

- To enter number of blocks and processes
- Run different algorithms
- Compare results in table

4. Working Of Algorithms And Functions:

This explains how different memory allocation algorithms and the supporting functions work in our system.

-first_fit() : Allocates the first available block.

-best_fit() : Allocates the smallest block.

-worst_fit() : Allocates the largest available block.

```
C: > Users > Admin > Downloads > DSA GAME.py > ...
1  import tkinter as tk
2  from tkinter import ttk
3  import random
4
5  # ----- Algorithms ----- #
6
7  def first_fit(blocks, processes):
8      alloc = [-1] * len(processes)
9      for i in range(len(processes)):
10         for j in range(len(blocks)):
11             if blocks[j] >= processes[i]:
12                 alloc[i] = j
13                 blocks[j] -= processes[i]
14                 break
15         return alloc
16
17  def best_fit(blocks, processes):
18      alloc = [-1] * len(processes)
19      for i in range(len(processes)):
20         best = -1
21         for j in range(len(blocks)):
22             if blocks[j] >= processes[i]:
23                 if best == -1 or blocks[j] < blocks[best]:
24                     best = j
25         if best != -1:
26             alloc[i] = best
27             blocks[best] -= processes[i]
28         return alloc
29
```

```

44 def worst_fit(blocks, processes):
45     alloc = [-1] * len(processes)
46     for i in range(len(processes)):
47         worst = -1
48         for j in range(len(blocks)):
49             if blocks[j] >= processes[i]:
50                 if worst == -1 or blocks[j] > blocks[worst]:
51                     worst = j
52         if worst != -1:
53             alloc[i] = worst
54             blocks[worst] -= processes[i]
55     return alloc

```

- calculate_waste() : Calculates memory wastage.
- fill_table() : Display results in the table.
- generate() : Creates random blocks and processes.

```

C: > Users > Admin > Downloads > DSA GAME.py > ...
44
45 def calculate_waste(blocks, alloc, processes):
46     temp = blocks.copy()
47
48     for i in range(len(processes)):
49         if alloc[i] != -1:
50             temp[alloc[i]] -= processes[i]
51
52     waste = 0
53     for i in range(len(blocks)):
54         if temp[i] != blocks[i]:
55             waste += temp[i]
56
57     return waste
58
59 def fill_table(table, processes, alloc):
60     for row in table.get_children():
61         table.delete(row)
62
63     for i in range(len(processes)):
64         block = f"B{alloc[i]+1}" if alloc[i] != -1 else "Not Allocated"
65         table.insert("", "end", values=(f"P{i+1}", processes[i], block))
66
67 def generate():
68     try:
69         nb = int(block_entry.get())
70         np = int(process_entry.get())
71     except:
72         return None, None

```

- run_first(), run_best(), run_worst() : Runs the individual algorithms.
- run_all() : Compare all algorithms.

```
96 def run_first():
97     b, p = generate()
98     if b:
99         show_single("First Fit", first_fit(b.copy(), p), p)
100
101 def run_best():
102     b, p = generate()
103     if b:
104         show_single("Best Fit", best_fit(b.copy(), p), p)
105
106 def run_worst():
107     b, p = generate()
108     if b:
109         show_single("Worst Fit", worst_fit(b.copy(), p), p)
110
111 def run_all():
112     blocks, processes = generate()
113     if not blocks:
114         return
115
116     hide_all()
117     tables_frame.pack(pady=15)
118
119     ff = first_fit(blocks.copy(), processes)
120     bf = best_fit(blocks.copy(), processes)
121     wf = worst_fit(blocks.copy(), processes)
122
123     fill_table(table_ff, processes, ff)
124     fill_table(table_bf, processes, bf)
125     fill_table(table_wf, processes, wf)
```

Output:

Memory Allocation Simulator

Blocks: Processes:

[First Fit](#) [Best Fit](#) [Worst Fit](#) [Compare All](#)

Memory Blocks: [348, 278, 198, 107]
Processes: [233, 182, 238, 80, 180]

First Fit		
PROCESSES	SIZES	BLOCK ALLOCATED
P1	233	B1
P2	182	B2
P3	238	Not Allocated
P4	80	B1
P5	180	B3

Memory Allocation Simulator

Blocks: Processes:

[First Fit](#) [Best Fit](#) [Worst Fit](#) [Compare All](#)

Memory Blocks: [143, 230, 387, 310]
Processes: [217, 128, 239, 192, 167]

Best Fit		
PROCESSES	SIZES	BLOCK ALLOCATED
P1	217	B2
P2	128	B1
P3	239	B4
P4	192	B3
P5	167	B3

Memory Allocation Simulator

Blocks: Processes:

First Fit

Best Fit

Worst Fit

Compare All

Memory Blocks: [322, 352, 112, 332]

Processes: [189, 216, 56, 97, 213]

Worst Fit

PROCESSES	SIZES	BLOCK ALLOCATED
P1	189	B2
P2	216	B4
P3	56	B1
P4	97	B1
P5	213	Not Allocated

Memory Allocation Simulator

Blocks: Processes:

First Fit

Best Fit

Worst Fit

Compare All

Memory Blocks: [373, 100, 208, 112]

Processes: [157, 114, 243, 130, 244]

Memory Wastage
First Fit = 180 | Best Fit = 67 | Worst Fit = 180

Most Efficient: Best Fit

First Fit

PROCESSES	SIZES	BLOCK ALLOCATED
P1	157	B1
P2	114	B1
P3	243	Not Allocated
P4	130	B3
P5	244	Not Allocated

Best Fit

PROCESSES	SIZES	BLOCK ALLOCATED
P1	157	B3
P2	114	B1
P3	243	B1
P4	130	Not Allocated
P5	244	Not Allocated

Worst Fit

PROCESSES	SIZES	BLOCK ALLOCATED
P1	157	B1
P2	114	B1
P3	243	Not Allocated
P4	130	B3
P5	244	Not Allocated

5. Features of the project

- It is user-friendly GUI
- Random generation of the memory blocks and processes
- Implementation of teen allocation techniques
- Comparison of the algorithms
- It displays the memory wastage
- It shows the most efficient algorithms

6. Conclusion

This project helps to understand that how a memory allocation works in an operating systems.

This shows that the different algorithms give the different results, and usually the best fit and the worst fit perform better to reduce the wastage.

7. Contribution

This work is divided between 3 members

Aisha: Implements the first fit and GUI integration

Areesha: Implements best fit algorithm and wastage calculation

Rameesha: Implements the worst fit algorithm and comparison logic.

8. References

- 1.<https://www.geeksforgeeks.org/memory-allocation-strategies>
- 2.https://www.tutorialspoint.com/operating_system/os_memory_management.htm